INFORME DE LABORATORIO #3

Diseño y Validación de Gramáticas

Antecedente	Información
Asignatura	Teoría de la Computación INFO1148
Carrera	Ingeniería Civil en Informática
Docente	Marcos Lévano
Integrantes	Juan Muñoz Veloso y Fernando Valdés Gómez
Fecha	20 de octubre de 2026

1. Introducción

En este laboratorio se diseña, analiza y valida una Gramática Libre de Contexto (GLC) capaz de representar la estructura sintáctica del lenguaje de programación simplificado del enunciado. Se identifican los terminales y los no terminales del lenguaje, se define formalmente la gramática $G = (V, \Sigma, P, S)$, se construyen sus producciones, se realizan derivaciones por la izquierda, se construye el árbol sintáctico de las cadenas solicitadas y se valida el comportamiento de la gramática frente a ocho casos, cuatro correctos y cuatro incorrectos. Como extensión creativa se incorpora la construcción de repetición (`mientras ... fin_mientras`) al lenguaje, lo que obliga a ampliar tanto el conjunto de terminales como el de no terminales. Toda la validación se apoya en un analizador descendente recursivo con retroceso entregado junto al informe. Además, las tres derivaciones principales se reprodujeron en JFLAP 7.1: el archivo `gramatica/GLC_Lab03.cfg` usa una codificación isomorfa de un carácter por símbolo, explicada en el anexo D, y las capturas de los árboles se incluyen como evidencia. La lectura del archivo se comprobó con la propia biblioteca de JFLAP 7.1; el detalle está en el anexo D.

2. Desarrollo

2.1 Pregunta 1 – Identificación formal de la gramática

a) Terminales y su clasificación

Los terminales son los símbolos que no admiten ninguna derivación: son las fichas concretas del lenguaje. Se clasifican según su función en el lenguaje.

Categoría	Terminales	Observación
Palabras reservadas	`Begin`, `End`, `Programa`, `salida`	No admiten otra función: el analizador léxico los reconoce como palabras completas.
Operadores aritméticos	`+`, `-`, `*`, `/`, `%`	Binarios. Cada expresión admite una sola operación, sin reglas de precedencia.
Operador de asignación	`=`	Único en el lenguaje; separa el identificador del valor.
Símbolos especiales	`#`	Termina asignaciones y salidas. El lazo se cierra con `fin_mientras`.
Identificadores	Clase léxica `IDENT`	Forma: `[A-Za-z][A-Za-z0-9_]*`. El guion bajo pertenece a la clase, no es un terminal aparte.
Números	Clase léxica `NUM`	Forma: `[0-9]+(\.[0-9]+)?`. Cubre `20`, `0` y `1.345`.

La extensión de la sección 3 agrega dos palabras reservadas (`mientras`, `fin_mientras`) y cuatro operadores relacionales (<, >, <=, >=), que se describen en detalle más adelante.

b) No terminales y función de cada uno

Los no terminales identificados son `<programa>`, `<lista_instrucciones>`, `<instruccion>`, `<asignacion>`, `<asignacion_simple>`, `<asignacion_expresion>`, `<valor>`, `<expresion>`, `<operando>`, `<operador>` y `<salida_instr>`. La extensión agrega `<mientras_instr>`, `<condicion>` y `<operador_rel>`. La función de cada uno se explica en el apartado 2.2 b).

c) Símbolo inicial

El símbolo inicial es `<programa>`. Toda cadena del lenguaje completo se obtiene derivando a partir de ese no terminal, que es el único que aparece en el encabezado `Begin Programa <id> ... End`. Las cadenas que son sólo un fragmento del programa (una instrucción suelta) se derivan a partir de `<instruccion>` o de `<lista_instrucciones>`, que son subgramáticas de la principal.

d) Definición formal $G = (V, \Sigma, P, S)$

Una GLC es un cuádruplo $G = (V, \Sigma, P, S)$: V contiene los no terminales, Σ los terminales, P las producciones y S el símbolo inicial. La definición siguiente incluye la extensión. La gramática base tiene 11 no terminales, 13 terminales y 20 producciones. La versión extendida agrega tres no terminales, seis terminales y siete producciones, para un total de 14, 19 y 27.

Componente	Definición
V	{ programa, lista_instrucciones, instruccion, asignacion, asignacion_simple, asignacion_expresion, valor, expresion, operando, operador, salida_instr, mientras_instr, condicion, operador_rel}, con $\mid V \mid = 14$.
Σ	{ Begin, End, Programa, salida, mientras, fin_mientras, +, −, *, /, %, =, #, <, >, <=, >=, IDENT, NUM}, con $\mid \Sigma \mid = 19$.
P	Conjunto de 27 reglas de producción, numeradas de P01 a P27 (sección 2.2).
S	`<programa>`.

Los conjuntos V y Σ son disjuntos y todas las reglas tienen la forma $X \to \alpha$ con $X \in V$, que es la condición de gramática libre de contexto.

e) Por qué no enumerar identificadores y números

Los identificadores y los números admiten infinitos valores. Enumerarlos exigiría infinitas producciones, mientras que una GLC debe tener un conjunto finito de reglas. Se reconocen por su forma durante el análisis léxico; la gramática sintáctica recibe las clases `IDENT` y `NUM`. También sería válido describirlos con reglas recursivas a nivel de caracteres, pero aquí se separan ambos niveles para mantener legibles las producciones.

La solución adoptada es la habitual en este tipo de lenguajes: los identificadores y los números se tratan como dos clases léxicas, `IDENT` y `NUM`, descritas por las expresiones regulares `[A-Za-z][A-Za-z0-9_]*` y `[0-9]+(\.[0-9]+)?`. La gramática trabaja con esas clases como si fueran fichas atómicas. En el anexo A se muestra cómo podría describirse la misma gramática en su versión carácter a carácter, con reglas del tipo `<identificador>` → `<letra> <resto_id>`, lo que confirma que la decisión es una simplificación legítima y no una omisión.

2.2 Pregunta 2 – Construcción de la Gramática Libre de Contexto

a) Producciones

La tabla siguiente contiene el conjunto completo de reglas P, con la justificación de cada una. En el lado derecho se omiten los ángulos para abreviar: los nombres definidos en V son no terminales; los restantes símbolos pertenecen a Σ. Las reglas P01 a P21, excepto P06, describen el lenguaje del enunciado; P06 y P22 a P27 corresponden a la extensión creativa de la sección 3 y se analizan allí.

Regla	Producción	Justificación
P01	`<programa>` → `Begin Programa IDENT lista_instrucciones End`	Encabezado del programa: la palabra `Programa` precede al nombre del programa y el cuerpo va entre `Begin` y `End`.
P02	`<lista_instrucciones>` → `instruccion`	Caso base: la lista puede tener una sola instrucción.

Regla	Producción	Justificación
P03	<lista_instrucciones> → instruccion lista_instrucciones	Caso recursivo: permite tantas instrucciones como se necesiten.
P04	<instruccion> → asignacion	Una instrucción puede ser una asignación.
P05	<instruccion> → salida_instr	Una instrucción puede ser una salida por pantalla.
P06	<instruccion> → mientras_instr	Extensión creativa: lazo repetitivo.
P07	<asignacion> → asignacion_simple	Distingue la asignación de un valor simple.
P08	<asignacion> → asignacion_expresion	Distingue la asignación de un resultado aritmético.
P09	<asignacion_simple> → IDENT = valor #	Asignación simple: identificador, igual, valor y terminador.
P10	<asignacion_expresion> → IDENT = expresion #	Asignación con operación aritmética.
P11	<valor> → NUM	Un valor puede ser un número literal.
P12	<valor> → IDENT	Un valor puede ser el contenido de otra variable (temporal = x #).
P13	<expresion> → operando operador operando	Expresión aritmética binaria, la forma que usa el programa de referencia.
P14	<operando> → IDENT	Un operando puede ser una variable.
P15	<operando> → NUM	Un operando puede ser una constante numérica.
P16	<operador> → +	Operador aritmético de suma.
P17	<operador> → -	Operador aritmético de resta.
P18	<operador> → *	Operador aritmético de multiplicación.
P19	<operador> → /	Operador aritmético de división.
P20	<operador> → %	Operador aritmético de módulo.
P21	<salida_instr> → salida IDENT #	Instrucción de salida: palabra reservada, identificador y terminador.

Regla	Producción	Justificación
P22	<mientras_instr> → mientras condicion lista_instrucciones fin_mientras	Extensión creativa: cabecera, condición, cuerpo y cierre del lazo.
P23	<condicion> → operando operador_rel operando	La condición compara dos operandos.
P24	<operador_rel> → <	Operador relacional menor que.
P25	<operador_rel> → >	Operador relacional mayor que.
P26	<operador_rel> → <=	Operador relacional menor o igual que.
P27	<operador_rel> → >=	Operador relacional mayor o igual que.

b) No terminales y función de cada uno

Los no terminales son los símbolos que se pueden derivar: representan las estructuras del lenguaje. La tabla siguiente justifica el papel de cada uno dentro de la gramática.

No terminal	Función
<programa>	Encabezado y cuerpo completo del programa: Begin Programa <id> ... End.
<lista_instrucciones>	Secuencia de una o más instrucciones del cuerpo del programa.
<instruccion>	Una instrucción del lenguaje: asignación, salida o lazo.
<asignacion>	Encabezamiento común de los dos tipos de asignación.
<asignacion_simple>	Asignación de un valor simple: <id> = <valor> #.
<asignacion_expresion>	Asignación del resultado de una operación: <id> = <expresion> #.
<valor>	Valor simple asignable: un número o un identificador.
<expresion>	Expresión aritmética binaria: <operando> <operador> <operando>.
<operando>	Cada uno de los dos operandos de una expresión.
<operador>	Operador aritmético binario: + - * / %.
<salida_instr>	Instrucción de salida por pantalla: salida <id> #.
<mientras_instr>	Extensión creativa: lazo mientras <condicion> ... fin_mientras.
<condicion>	Extensión creativa: comparación entre dos operandos.
<operador_rel>	Extensión creativa: operador relacional < > <= >=.

Dos nombres llevan un sufijo (<salida_instr>, <mientras_instr>) porque la terminal salida y la terminal mientras existen en Σ. La condición formal $V \cap \Sigma = \emptyset$ exige que un no terminal y un terminal no se llamen igual, de modo que se distinguen los símbolos <salida_instr> y <mientras_instr> de las palabras reservadas correspondientes.

c) Diferenciación entre asignación simple, asignación con expresión y salida

El enunciado pide diferenciar explícitamente las tres construcciones. La diferencia no es de nombre, sino de posición dentro de la regla y de la forma del lado derecho:

- **Asignación simple (P09):** `<asignacion_simple>` → `IDENT = valor #`. El valor es un único operando simple, un número o un identificador. Ejemplos: `x = 20 #`, `temporal = x #`.
- **Asignación con expresión aritmética (P10):** `<asignacion_expresion>` → `IDENT = expresion #`, donde `<expresion>` → `<operando> <operador> <operando>` exige obligatoriamente un operador binario. Ejemplos: `z = x + y #`, `k = h * y #`.
- **Instrucción de salida (P21):** `<salida_instr>` → `salida IDENT #`. No hay signo igual ni expresión: el argumento es directamente un identificador. Ejemplo: `salida z #`.

La asignación simple y la asignación con expresión se distinguen porque la segunda exige un operador binario. En los casos ensayados, cada cadena aceptada tiene un único árbol sintáctico. Este conteo comprueba los ejemplos, pero por sí solo no demuestra la ausencia de ambigüedad para todas las cadenas del lenguaje.

d) Verificación con el programa de referencia

El programa de referencia del enunciado se reconoce completo: el analizador lo acepta como derivado de `<programa>`. La derivación por la izquierda tiene 108 pasos; las formas sentenciales comienzan así:

```
<programa>
⇒ Begin Programa operativo_01_02 <lista_instrucciones> End
⇒ Begin Programa operativo_01_02 <instruccion> <lista_instrucciones> End
⇒ Begin Programa operativo_01_02 <asignacion> <lista_instrucciones> End
⇒ Begin Programa operativo_01_02 <asignacion_simple> <lista_instrucciones> End
⇒ Begin Programa operativo_01_02 x = <valor> # <lista_instrucciones> End
⇒ Begin Programa operativo_01_02 x = 1.345 # <lista_instrucciones> End
⇒ Begin Programa operativo_01_02 x = 1.345 # <instruccion> <lista_instrucciones>
End
⇒ ...
```

La derivación completa está en el archivo indicado en el anexo B y en el archivo `evidencias/evidencias_lab03.txt`. El programa contiene 91 fichas (`token`), contando las palabras reservadas, los identificadores, los números, los operadores y los terminadores.

2.3 Pregunta 3 – Derivaciones y árbol sintáctico

a) Derivación por la izquierda de `z = x + y #`

La cadena pedida es una instrucción, por lo que la derivación se inicia en `<instruccion>`, el no terminal que la genera dentro de `<lista_instrucciones>`. Cada paso reemplaza el no terminal más a la izquierda de la forma sentencial, que es lo que caracteriza a una derivación por la izquierda. Se usa notación con lexemas: al aplicar una regla con `IDENT` o `NUM`, se muestra directamente el texto del token reconocido. Así, `z`, `x` e `y` son ocurrencias de `IDENT`; no son nuevos terminales ni requieren reglas adicionales. Los árboles siguen la misma convención en sus hojas.

```
<instruccion>
⇒ <asignacion>   [P04]
⇒ <asignacion_expresion>   [P08]
⇒ z = <expresion> #   [P10]
⇒ z = <operando> <operador> <operando> #   [P13]
⇒ z = x <operador> <operando> #   [P14]
⇒ z = x + <operando> #   [P16]
⇒ z = x + y #   [P14]
```

b) Derivación por la izquierda de `salida z #`

```
<instruccion>
⇒ <salida_instr>   [P05]
⇒ salida z #   [P21]
```

La derivación es más corta porque la instrucción de salida no tiene partes que expandir: los tres símbolos de la derecha de P21 son ya terminales. El árbol adicional de esta cadena se incluye en el anexo E.

c) Árbol sintáctico de `z = x + y #`

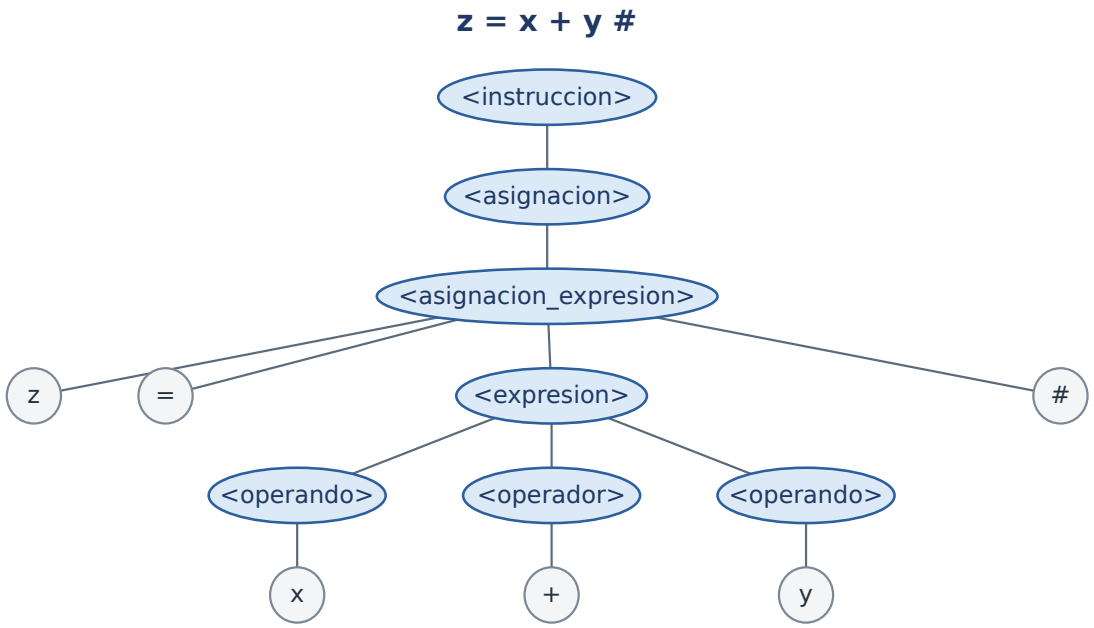


Figura 1. Árbol de derivación de la cadena `z = x + y #` obtenido con la herramienta de validación entregada junto al informe.

La raíz es `<instruccion>`; los nodos internos son los no terminales que aún debían expandirse y las hojas son los terminales leídos, de izquierda a derecha: `z`, `=`, `x`, `+`, `y` y `#`.

d) Relación entre la raíz, los no terminales, los terminales y las hojas

La raíz `<instruccion>` identifica la estructura que se deriva. Los nodos internos son no terminales y sus hijos representan el lado derecho de la producción aplicada. Las hojas son terminales; para

IDENT y NUM se muestra el lexema del token. Leer las hojas de izquierda a derecha reconstruye z = x + y #. El árbol conserva la jerarquía de la derivación, mientras que los pasos por la izquierda indican en qué orden se expandieron los no terminales.

e) Validación de las tres cadenas adicionales

Las tres cadenas de la Pregunta 3.14 se generan con la gramática propuesta:

Cadena	¿Generable?	Justificación
x = 20 #	Sí	Asignación simple (P09): <valor> → NUM, cuyo lexema es 20.
z = x * y #	Sí	Asignación con expresión (P10): * se deriva con P18 y ambos operandos son identificadores.
salida temporal #	Sí	Instrucción de salida (P21) con el identificador temporal.

2.4 Pregunta 4 – Validación y detección de errores

a) Método de validación

La validación se realizó con un analizador descendente recursivo con retroceso completo, entregado en el archivo herramientas/validar_gramatica.py. El analizador tokeniza el código fuente (ignorando los comentarios /* ... */), aplica exactamente las 27 reglas de producción entregadas y devuelve, cuando la cadena es válida, el árbol sintáctico correspondiente. El motor de análisis de JFLAP 7.1 trabaja carácter a carácter: las letras mayúsculas son variables y cualquier otro carácter es terminal. Para que el archivo gramatica/GLC_Lab03.cfg represente la misma gramática, cada símbolo se codifica con un solo carácter (anexo D). El archivo se cargó con la propia biblioteca de JFLAP 7.1 y reporta 14 variables, 19 terminales y 27 producciones, con S como símbolo inicial (anexo D). Las capturas del anexo D muestran las derivaciones de las tres cadenas principales en JFLAP.

b) Resultado del análisis de los ocho casos

Caso	Cadena	Resultado	Análisis
1	x = 10 #	Aceptada	Asignación simple: P09 con <valor> → NUM.
2	z=x+y#	Aceptada	Asignación con expresión: P10, P13 y P16.
3	salida z #	Aceptada	Instrucción de salida: P05 y P21.
4	x 10 #	Rechazada	Falta el operador de asignación =: la regla P09 exige la secuencia IDENT = valor #.
5	z=+xy#	Rechazada	El signo + queda donde la gramática espera un operando: P13 exige <operando> <operador> <operando> y el lenguaje no admite signos unarios.
6	salida #	Rechazada	Falta el argumento de la instrucción: P21 exige salida IDENT #.

Caso	Cadena	Resultado	Análisis
7	`x = 20`	Rechazada	Falta el terminador de instrucción `#`.
8	`Begin Programa prueba x = 5 # salida x # End`	Aceptada	Programa completo: P01 con dos instrucciones en el cuerpo.

c) Alcance de los rechazos

El caso 5 tiene un error sintáctico, no léxico: `xy` sí es un identificador válido, pero `+` no puede aparecer antes del primer operando. En los casos 4, 6 y 7 faltan, respectivamente, `=`, el identificador de salida y `#`. El análisis comprueba la estructura; no comprueba declaración de variables, tipos ni divisiones por cero.

3. Extensión de la gramática

De las cinco opciones propuestas se eligió la **opción B, repetición**, porque introduce una construcción que anida otras instrucciones y obliga a ampliar la gramática en los dos sentidos: nuevos terminales y nuevos no terminales.

a) Sintaxis de la construcción

> `mientras` *comparación instrucciones* `fin_mientras`

La comparación sigue el patrón `<operando> <operador_rel> <operando>`, con los operadores relacionales `<`, `>`, `<=` y `>=`. El cuerpo del lazo es una lista de instrucciones con la misma sintaxis del programa, por lo que puede contener asignaciones, salidas e incluso otros lazos.

b) Terminales y no terminales nuevos

Se agregan a Σ los terminales `mientras` y `fin_mientras` (palabras reservadas) y `<`, `>`, `<=`, `>=` (operadores relacionales). Se agregan a V los no terminales `<mientras_instr>`, `<condicion>` y `<operador_rel>`. Las reglas nuevas son:

Regla	Producción
P06	`<instruccion>` $\rightarrow$ `mientras_instr`
P22	`<mientras_instr>` $\rightarrow$ `mientras condicion lista_instrucciones fin_mientras`
P23	`<condicion>` $\rightarrow$ `operando operador_rel operando`
P24	`<operador_rel>` $\rightarrow$ `<`
P25	`<operador_rel>` $\rightarrow$ `>`
P26	`<operador_rel>` $\rightarrow$ `<=`
P27	`<operador_rel>` $\rightarrow$ `>=`

Con P06 el lazo pasa a ser una instrucción más y, por tanto, puede aparecer en el cuerpo del programa y también dentro del cuerpo de otro lazo. La condición reutiliza `<operando>`, lo que permite comparar variables y constantes numéricas. No admite expresiones aritméticas como operandos de la comparación.

c) Cadena válida y cadena inválida

Caso	Cadena
Válida	`Begin Programa lazo mientras x < 10 x = x + 1 # fin_mientras salida x # End`
Inválida	`Begin Programa lazo mientras x < 10 x = x + 1 # salida x # End`

La cadena inválida omite la palabra reservada `fin_mientras`: el lazo queda abierto y el analizador rechaza la cadena, porque P22 exige que al cuerpo del lazo le siga `fin_mientras`. Una segunda forma de error sería un operador relacional ausente, como en `mientras x 10 ... fin_mientras`.

d) Derivación de ejemplo

La derivación por la izquierda de la instrucción de lazo se obtiene del mismo modo que las anteriores:

Se usan estas abreviaturas tipográficas de los mismos no terminales:

L: `<lista_instrucciones>`; I: `<instruccion>`.
A: `<asignacion>`; E: `<asignacion_expresion>`.
X: `<expresion>`; O: `<operando>`; R: `<operador>`.

Regla	Forma sentencial
Inicio	I
P06	`<mientras_instr>`
P22	`mientras <condicion>` L `fin_mientras`
P23	`mientras` O `<operador_rel>` O L `fin_mientras`
P14	`mientras x <operador_rel>` O L `fin_mientras`
P24	`mientras x <` O L `fin_mientras`
P15	`mientras x < 10` L `fin_mientras`
P02	`mientras x < 10` I `fin_mientras`
P04	`mientras x < 10` A `fin_mientras`
P08	`mientras x < 10` E `fin_mientras`
P10	`mientras x < 10 x =` X `# fin_mientras`
P13	`mientras x < 10 x =` O R O `# fin_mientras`
P14	`mientras x < 10 x = x` R O `# fin_mientras`
P16	`mientras x < 10 x = x +` O `# fin_mientras`
P15	`mientras x < 10 x = x + 1 # fin_mientras`

e) Árbol sintáctico de la extensión creativa

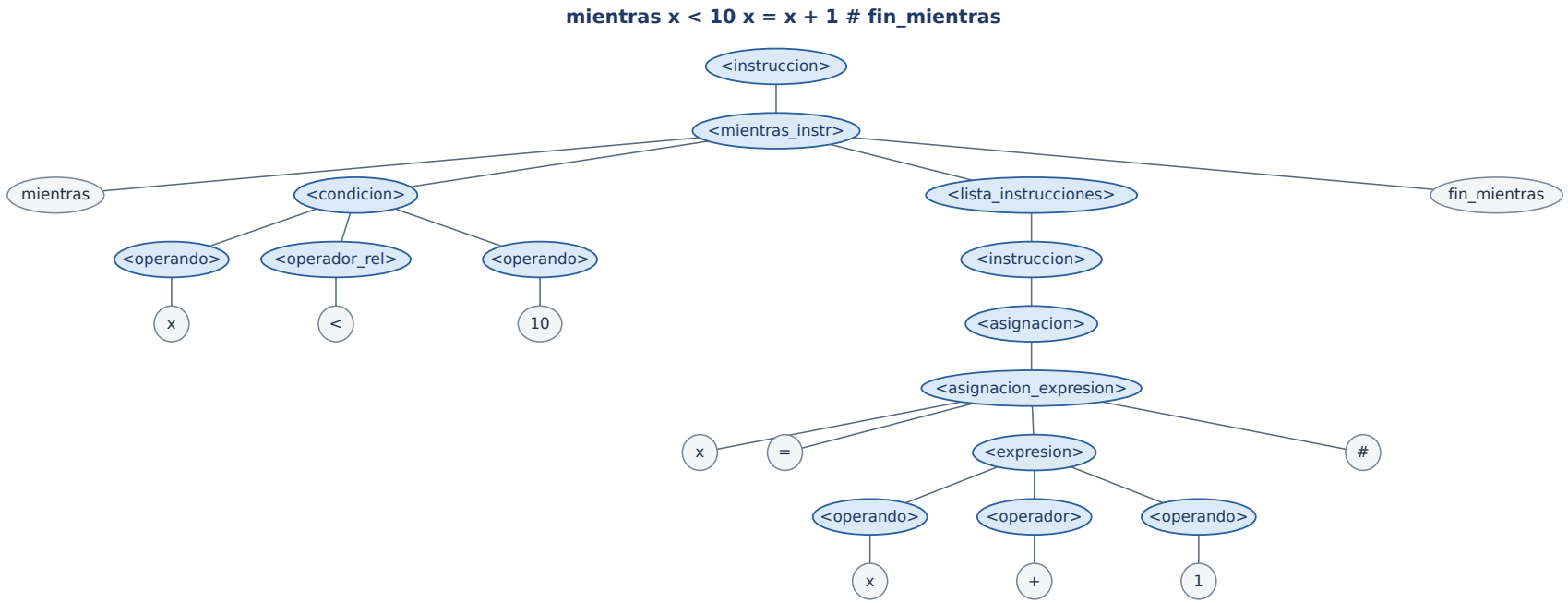


Figura 2. Árbol de derivación de la instrucción de lazo `mientras x < 10 x = x + 1 # fin_mientras`.

La raíz `<instruccion>` se expande con P06 hasta `<mientras_instr>`, que, a su vez, se expande con P22. Se distinguen las cuatro partes de la construcción: la palabra reservada `mientras`, la condición `<condicion>`, el cuerpo `<lista_instrucciones>` y el cierre `fin_mientras`.

4. Conclusiones

La gramática extendida describe el lenguaje del enunciado con 14 no terminales, 19 terminales y 27 producciones, con `<programa>` como símbolo inicial. Tratar los identificadores y los números como clases léxicas permite mantener un conjunto finito de reglas y separar el análisis léxico del sintáctico. Las producciones distinguen la asignación simple, la asignación con expresión y la salida. La validación acepta el programa de referencia completo y rechaza los cuatro casos incorrectos del enunciado; cada cadena aceptada en las pruebas tiene un único árbol sintáctico. La extensión `mientras ... fin_mientras` incorpora repetición y anidamiento mediante nuevas reglas que reutilizan los operandos y las listas de instrucciones de la gramática base.

JFLAP 7.1 trabaja con símbolos de un carácter, por lo que se definió una codificación isomorfa para representar la misma gramática en el archivo entregado. Su biblioteca comprobó la lectura de las 27 producciones y las capturas del anexo D documentan las tres derivaciones principales. El analizador propio valida las cadenas por tokens y genera los árboles legibles. Estas pruebas comprueban la estructura sintáctica de los casos ensayados; no comprueban declaración de variables, tipos ni divisiones por cero, y tampoco constituyen una demostración general de ausencia de ambigüedad.

5. Consideraciones de entrega

La gramática para JFLAP se encuentra en GitHub mediante el enlace directo a `GLC_Lab03.cfg`. El archivo usa la codificación de un carácter por símbolo descrita en el anexo D.

La carpeta de entrega contiene el informe y los siguientes archivos:

- `Lab03_Juan_Munoz_Veloso.pdf`: informe del laboratorio.
- `gramatica/GLC_Lab03.cfg`: gramática en el formato XML de JFLAP 7.1, con la codificación isomorfa de un carácter por símbolo (14 variables, 19 terminales, 27 producciones).
- `gramatica/GLC_Lab03.txt`: definición formal legible de $G = (V, \Sigma, P, S)$ y tabla de codificación para JFLAP.
- `herramientas/gramatica_lab3.py`: especificación de la gramática (fuente única de verdad).
- `herramientas/validar_gramatica.py`: analizador, generador de derivaciones y de árboles.
- `herramientas/generar_cfg_jflap.py`: generador del archivo de JFLAP y de la tabla de codificación.
- `herramientas/VerificarGramaticaJFLAP.java`: verificador del archivo de JFLAP con la biblioteca de JFLAP 7.1.
- `herramientas/CapturarArbolJFLAP.java`: harness que reproduce las capturas de los árboles con JFLAP 7.1.
- `figuras/`: árboles sintácticos en PDF vectorial, con originales SVG y copias PNG, y capturas de JFLAP 7.1 en `jflap_*.png`.
- `evidencias/evidencias_lab03.txt` y `evidencias/verificacion_jflap.txt`: registro de validación propia y de verificación del archivo de JFLAP, respectivamente.
- `README.md`: guía de la carpeta de entrega y de cómo reproducir las evidencias.

Las figuras 1, 6 y 2 son esquemas legibles generados por el analizador propio; las figuras 3, 4 y 5 son capturas de JFLAP 7.1 de las mismas cadenas. Las evidencias adicionales se reúnen en los anexos C, D y E.

Anexos

Anexo A – La gramática en su versión carácter a carácter

La versión entregada trata los identificadores y los números como clases léxicas. Si se quisiera llevar esa descripción al interior de la gramática, bastaría reemplazar `IDENT` y `NUM` por reglas como `<identificador>` → `<letra> <resto_id>`, `<resto_id>` → `<resto_id> <letra>` | `<resto_id> <digito>` | `<resto_id> _` | ε, y `<numero>` → `<entero>` | `<entero> . <entero>`, `<entero>` → `<digito> <entero>` | `<digito>`. Esa versión exige enumerar las 52 letras y los 10 dígitos, lo que multiplica el tamaño de la gramática sin aportar capacidad descriptiva: es la razón por la que se prefirió la versión con clases léxicas.

Anexo B – Derivación completa del programa de referencia

El archivo `evidencias/evidencias_lab03.txt` contiene, en su sección 5, las 108 aplicaciones de producciones.

Anexo C – Registro de la validación automática

El mismo archivo, en sus secciones 2, 2.b, 3, 6 y 7, registra el veredicto de los ocho casos, el número de árboles sintácticos de cada uno (uno en todos los casos aceptados, lo que confirma análisis único en esos ejemplos), el análisis de las cadenas adicionales y los resultados de la extensión creativa.

Cadenas de control adicionales

Para reforzar la validación se analizaron otras 24 cadenas, incluidos tres controles Unicode; la tabla muestra las más representativas. Las rechazadas confirman, cada una, una restricción del lenguaje.

Cadena	Resultado	Observación
`x = 1.345 #`	Aceptada	Número real con punto decimal.
`m = k% 10 #`	Aceptada	Operador módulo (P20).
`t = k/z#`	Aceptada	División sin espacios (P19).
`temporal = x #`	Aceptada	Asignación del valor de otra variable (P12).
`z = x + y * z #`	Rechazada	Expresión con dos operadores: la gramática define la operación binaria del enunciado.
`Begin Programa p End`	Rechazada	Cuerpo vacío: la lista de instrucciones exige al menos una (P02, P03).
`x == 20 #`	Rechazada	Doble signo igual: no existe la producción que lo permita.
`9x = 20 #`	Rechazada	Un identificador no puede empezar con un dígito.

Cadena	Resultado	Observación
`salida 20 #`	Rechazada	La salida exige un identificador, no un número.
`SALIDA x #`	Rechazada	Las palabras reservadas distinguen mayúsculas.
`begin Programa p x = 1 # End`	Rechazada	`Begin` es la única forma de abrir un programa.

Anexo D – Verificación y capturas de JFLAP 7.1

JFLAP 7.1 no acepta terminales de varios caracteres: su analizador recorre el lado derecho de cada producción carácter a carácter y considera variable a toda letra mayúscula. Para que el archivo entregado represente la misma gramática sin cambiar el lenguaje, se usa una codificación isomorfa de un carácter por símbolo. La tabla siguiente muestra el mapeo completo; la gramática formal del informe sigue usando los nombres legibles.

No terminal	Código	Terminal	Código
`<programa>`	`S`	`Begin`	`b`
`<lista_instrucciones>`	`L`	`End`	`e`
`<instruccion>`	`I`	`Programa`	`p`
`<asignacion>`	`A`	`salida`	`s`
`<asignacion_simple>`	`B`	`mientras`	`m`
`<asignacion_expresion>`	`E`	`fin_mientras`	`f`
`<valor>`	`V`	`+`	`+`
`<expresion>`	`X`	`-`	`-`
`<operando>`	`O`	`*`	`*`
`<operador>`	`R`	`/`	`/`
`<salida_instr>`	`Q`	`%`	`%`
`<mientras_instr>`	`M`	`=`	`=`
`<condicion>`	`C`	`#`	`#`
`<operador_rel>`	`T`	`<`	`<`
Terminales adicionales			
		`>`	`>`
		`<=`	`l`
		`>=`	`g`
		`IDENT`	`i`
		`NUM`	`n`

Con esa codificación, JFLAP lee 14 variables, 19 terminales y 27 producciones, y el símbolo inicial es `S`, que corresponde a `<programa>`. La salida del verificador está en `evidencias/verificacion_jflap.txt`. Las tres derivaciones se reproducen con las cadenas codificadas de la tabla siguiente; en cada captura se indica el no terminal desde el que se inicia la derivación, de modo que el árbol coincida con el esquema legible de las figuras 1, 6 y 2.

Cadena legible	Cadena codificada	Inicio
`z = x + y #`	`i=i+i#`	`<instruccion>`
`salida z #`	`si#`	`<instruccion>`
`mientras x < 10 x = x + 1 # fin_mientras`	`mi<ni=i+n#f`	`<mientras_instr>`

a) Captura de JFLAP 7.1 para `z = x + y #`

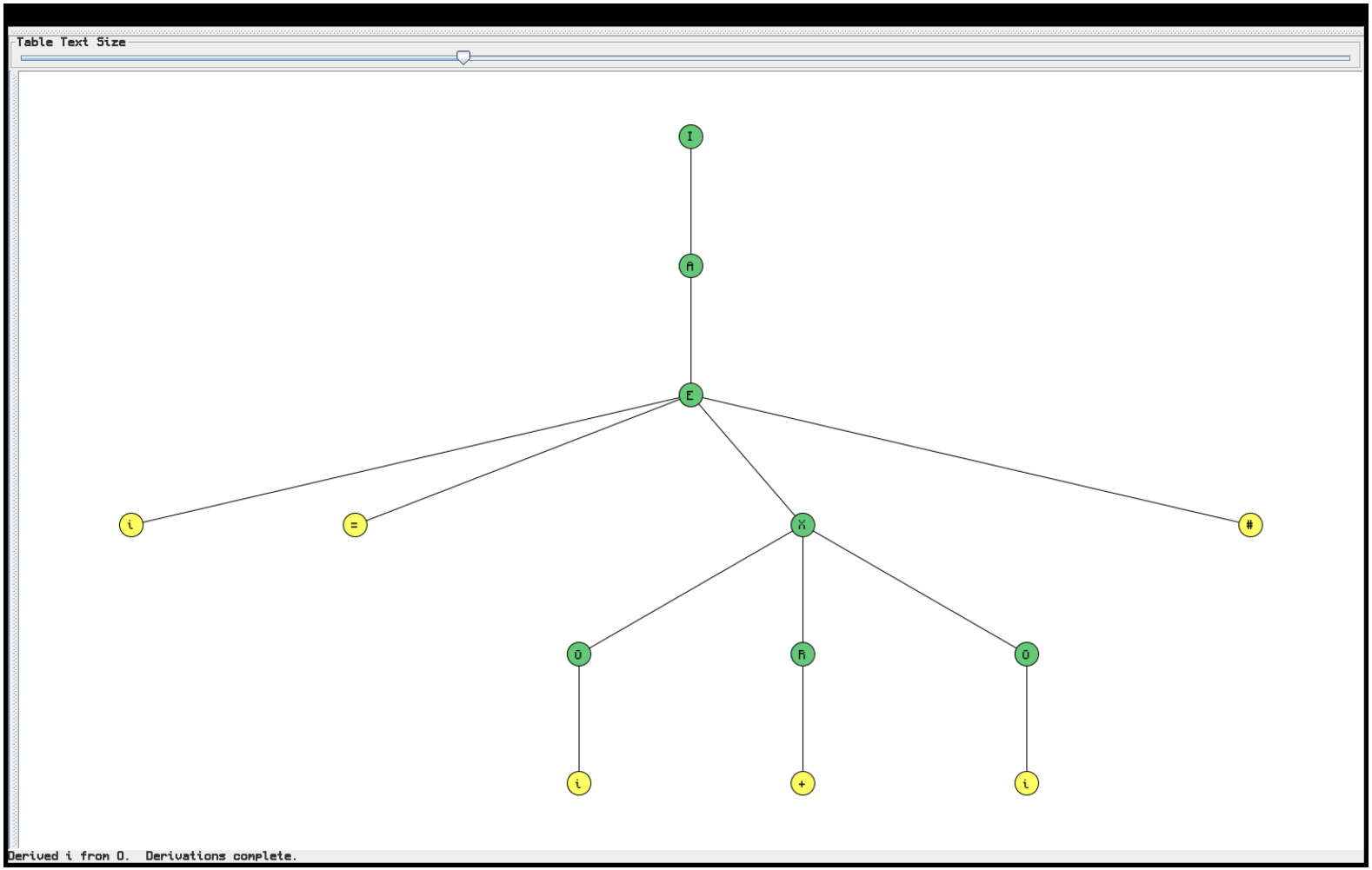


Figura 3. Árbol de derivación producido por JFLAP 7.1 al analizar la cadena codificada `i=i+i#` desde `<instruccion>`.

La raíz es `I`; las hojas `i`, `=`, `i`, `+`, `i` y `#` corresponden a `z`, `=`, `x`, `+`, `y` y `#`. La barra inferior muestra el estado final de JFLAP: `Derivations complete`.

b) Captura de JFLAP 7.1 para `salida z #`

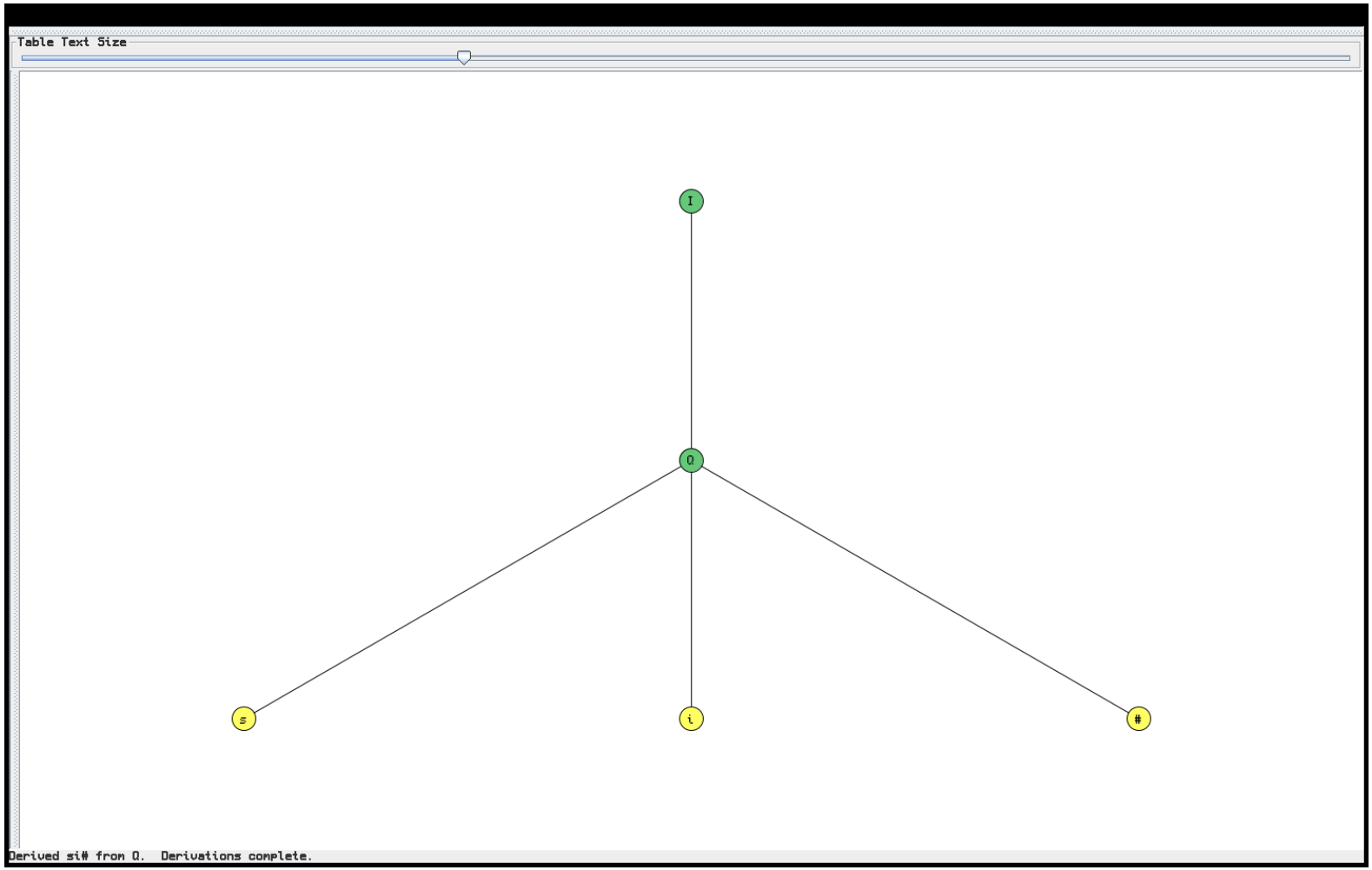


Figura 4. Árbol de derivación de la cadena codificada `si#` desde `<instruccion>`.

La raíz es `I`, que se expande con P05 hasta `Q`; las hojas `s`, `i` y `#` son `salida`, `z` y `#`.

c) Captura de JFLAP 7.1 para la extensión creativa

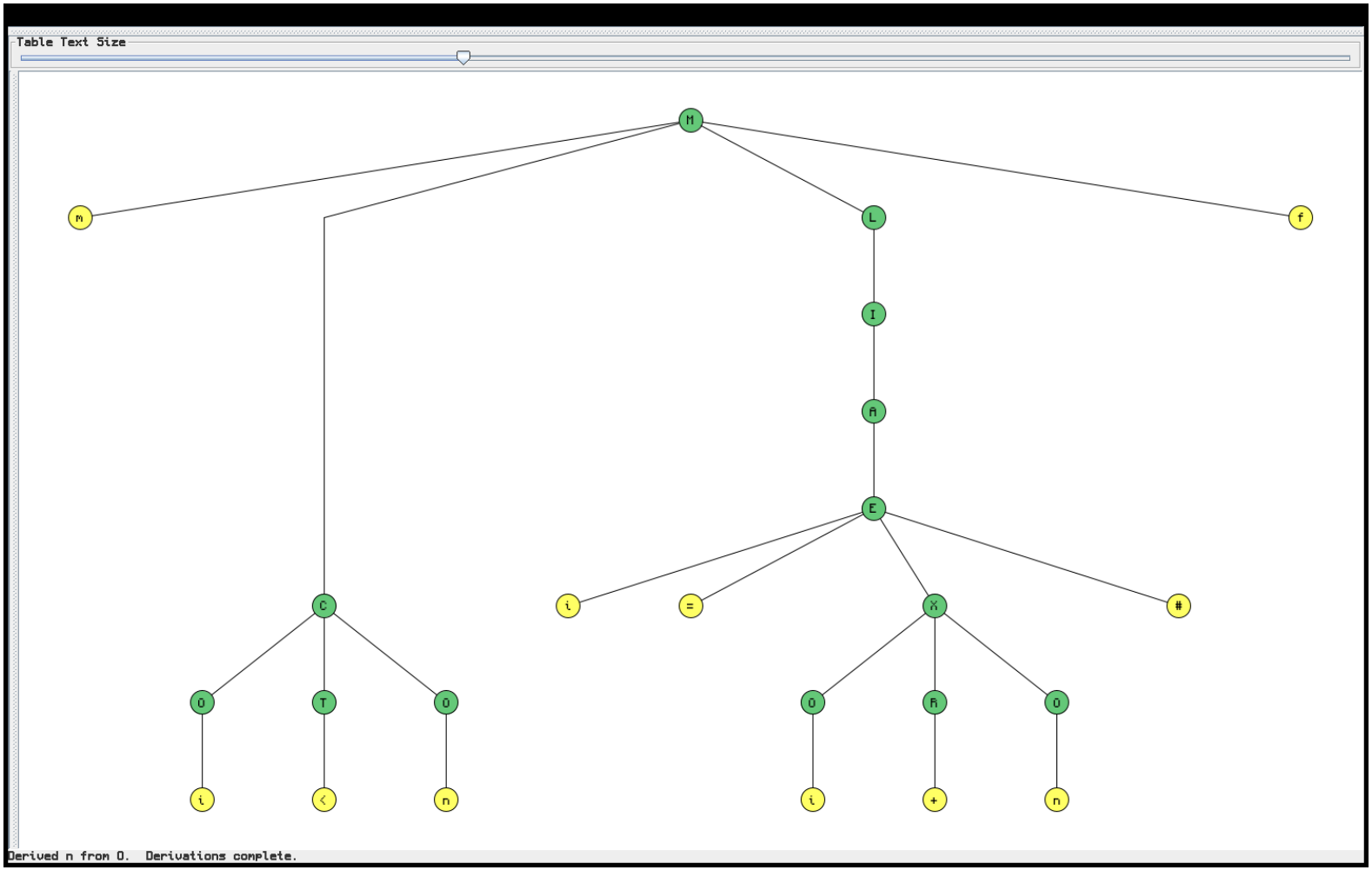


Figura 5. Árbol de derivación de la cadena codificada `mi<ni=i+n#f` desde `<mientras_instr>`.

La raíz `M` se expande con P22 y reproduce las cuatro partes del lazo: `mientras`, `<condicion>`, `<lista_instrucciones>` y `fin_mientras`. Esta captura ejercita P22, P23 y P24; las producciones P25–P27 quedan definidas, pero el ejemplo no usa >, <= ni >=.

Anexo E – Árbol adicional de la instrucción de salida

Árbol sintáctico de `salida z #`

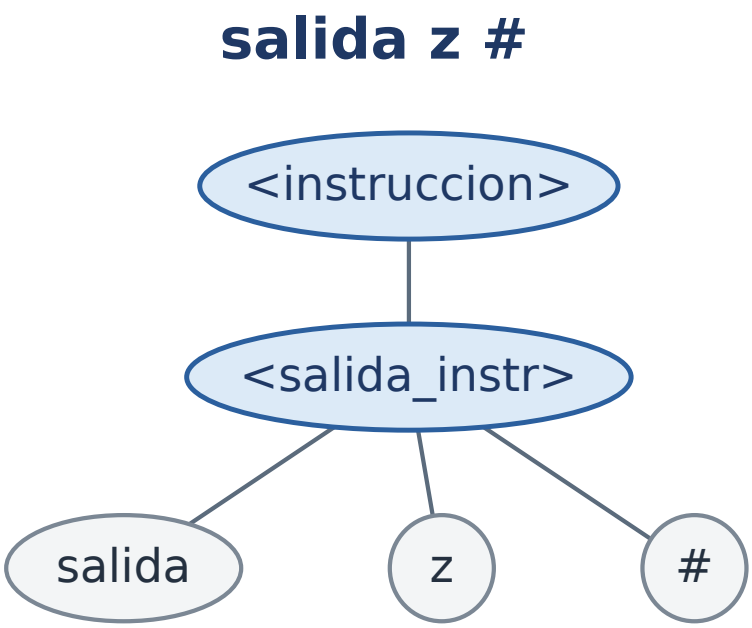


Figura 6. Árbol de derivación de la cadena `salida z #`.

La raíz `<instruccion>` se expande con P05 hasta `<salida_instr>`, que a su vez se expande con P21. Las tres hojas son los terminales `salida`, `z` y `#`.